CSCE 638: Human Chess Move Prediction for Predictive Caching

Hung Nguyen
737002625
nguye3hv@tamu.edu

Mukil Senthilkumar
836000195
mukilsenthil@tamu.edu

Seyed Ali Ghazi Asgar
532007084
alighazi@tamu.edu

Arif Nizami
137009847
arifnizami003@tamu.edu

1 Introduction

Chess has long served as a benchmark for artificial intelligence, with modern systems combining powerful search and learned evaluation (Silver et al., 2017; The Stockfish Developers, 2024). Recent studies have also explored using language models to represent chess states and generate moves from textual encodings such as PGN or FEN (Feng et al., 2023; Kolasani et al., 2025). These developments motivate exploring whether large language models can be used not only to generate chess moves, but also to predict the moves that human players are likely to choose.

Caching is a fundamental optimization technique widely used in operating systems, web services, and user-facing applications to reduce latency and improve responsiveness. In interactive chess applications, caching can be useful when likely future positions are known in advance. If a system can anticipate the opponent's next move, it can precompute an engine response before the move is actually played. Then, when the predicted move matches the player's actual move, the system can return a cached response immediately instead of waiting for a new engine computation. This makes move prediction useful not only as a modeling task, but also as a practical mechanism for reducing user-perceived latency.

Although chess engines are highly effective at identifying strong moves, they are not explicitly designed to predict what human players will actually choose. This creates a gap between engine-based move ranking and human-centered move prediction. In many practical chess applications, especially interactive platforms, the most useful prediction is not necessarily the objectively best move, but the move that is likely to occur next. This distinction motivates our project: we treat chess move prediction as a behavioral prediction problem whose output can be used to support faster system responses.

The novelty of our work is that we connect human chess move prediction with predictive caching. Instead of evaluating move prediction only as a standalone accuracy task, we study whether predicted human moves can be used to precompute engine responses and reduce user-perceived latency. Our contribution is a hybrid framework in which a fine-tuned language model predicts likely human moves from board state, legal moves, and Elo information, while Stockfish computes responses for the predicted branches. This allows us to evaluate both prediction quality and practical latency benefits in the same system.

In this project, we propose a chess move prediction system that anticipates likely human moves and precomputes candidate responses. Given the current board state, move history, and player skill information such as Elo rating, the system predicts the opponent's top-k next moves and uses a chess engine to compute responses in advance. If the actual move matches one of the predictions, the system can return a cached response immediately. Our results show that the fine-tuned language model improves human move prediction over Stockfish baselines in several settings, especially for top-1 and top-3 prediction, although its prediction latency is higher than Stockfish at depth 10.

2 Related Work

Chess AI has evolved from classical search-based engines to neural and language-model-based systems. Search-driven engines such as Stockfish and self-play systems such as AlphaZero demonstrate that high-level chess performance can be achieved through strong search, learned evaluation, and large-scale reinforcement learning (Silver et al., 2017; The Stockfish Developers, 2024). More recently, language-model-based approaches have shown that chess states and move sequences can be represented as structured text using encodings such as Portable Game Notation (PGN) and Forsyth–Edwards Notation (FEN), making chess a useful testbed for sequence modeling and reasoning with large language models (Feng et al., 2023; Kolasani et al., 2025). These developments provide the technical foundation for using LLMs not only to analyze chess positions, but also to predict plausible future moves.

A separate and increasingly important research

direction focuses on modeling human chess behavior rather than engine-optimal play. Maia-2 and related work show that predicting moves chosen by human players requires a different objective from maximizing engine strength, since human decisions reflect skill level, style, and imperfect reasoning (Tang et al., 2024; Rokach and Shapira, 2026). More recent work has continued this direction by studying skill-aware human move prediction and by investigating the use of large language models for anticipating human chess actions (Kreiger, 2024). This line of research suggests that human next-move prediction is best understood as a behavioral modeling problem, where the target is not the best move in theory, but the move most likely to be chosen in practice.

Another relevant line of work explores hybrid systems that combine language-model capabilities with traditional chess engines. Prior studies have used language models to interpret chess states, generate chess-related outputs, or complement engine-based analysis rather than replace it entirely (Feng et al., 2023; Libralesso, 2025). This hybrid perspective is important because engines remain much stronger at tactical search and move evaluation, while language models may be better suited to capturing human-like preferences or patterns. Our work follows this broader direction by separating the roles of prediction and response generation: the language model forecasts likely human moves, while the engine computes strong responses for those predicted branches.

Beyond chess, predictive computation has long been studied as a way to reduce perceived delay in interactive systems. In gaming and networked environments, prior work has shown that forecasting likely user actions and preparing responses ahead of time can compensate for latency and improve responsiveness (Halbhuber et al., 2022; Motoo et al., 2021; Gutmann and Konagaya, 2019). More generally, recent systems work has explored learned predictive caching strategies that use historical patterns to anticipate future requests and reduce access cost. These ideas are closely related to our setting, since a predicted chess move can be treated as a forecasted future action whose corresponding response can be prepared in advance.

Despite this progress, an important gap remains in the literature. Existing chess research typically studies either engine-strength move generation, human move prediction, or language-model-based chess reasoning in isolation. Likewise, prior predictive-latency and caching work demonstrates the value of anticipation in interactive systems, but does not examine a chess-specific framework where top-k human move forecasts are used to trigger precomputed engine responses. What is missing is a unified system that connects human-centered move prediction with practical latency reduction.

Our project addresses this gap by reframing chess move prediction as a practical anticipatory systems problem rather than only a modeling benchmark. Instead of evaluating whether a model can merely generate plausible chess moves, we study whether human next-move forecasts can be used to support precomputation of engine responses before the actual move is observed. This changes the role of prediction: the objective is no longer just to approximate human behavior, but to make that approximation operationally useful in a latency-sensitive pipeline.

More specifically, our approach combines legality-aware prompting, player metadata, and top-k move forecasting to produce a set of likely human actions that can directly drive predictive caching. The language model is used to capture human-like decision tendencies, while Stockfish remains responsible for computing tactically strong responses for the predicted branches. In this way, the project bridges two lines of work that are usually treated separately: human-centered chess prediction and latency-reduction through anticipatory computation. By evaluating both cache hit behavior and user-perceived latency, our work shows how human move prediction can serve as a systems mechanism for improving responsiveness, not just as an isolated prediction task.

3 Methodology

3.1 Problem Definition

We formulate the task as top-k human next-move prediction for predictive caching. Let s_t denote the current board state at time t, L_t denote the set of legal moves available in the current position, and e denote player metadata such as Elo ratings. Given (s_t, L_t, e), the model predicts a candidate set of likely human next moves

$$M_k = \{\hat{m}_1, \hat{m}_2, \ldots, \hat{m}_k\}.$$

For each predicted move in M_k, the system precomputes a corresponding engine response and stores it in cache. If the true opponent move

$m_t \in M_k$, the cached response is returned immediately; otherwise, the system falls back to standard on-demand engine computation. This formulation connects human move forecasting with predictive computation, where the quality of the predicted set determines whether precomputed responses can reduce perceived delay (Motoo et al., 2021; Gutmann and Konagaya, 2019; Halbhuber et al., 2022).

3.2 Proposed Approach

Our method uses a legality-aware prompting strategy together with parameter-efficient fine-tuning of a pretrained language model. Rather than allowing unrestricted move generation, we provide the model with the current board representation, player metadata, and the complete legal-move list, and prompt it to predict the next move from that set. This design makes the prediction space well-defined and encourages outputs that are directly usable in the caching pipeline. The use of textual chess state representations is consistent with prior work that models chess positions and move sequences using formats such as PGN and FEN for language-model-based reasoning (Feng et al., 2023; Kolasani et al., 2025).

The language model used in our implementation is Qwen3.5-9B. The model is fine-tuned on the training split using QLoRA with position-level prompts and the actual human move as the target output. QLoRA is a parameter-efficient fine-tuning approach that keeps the base model frozen and updates only a small set of low-rank adapter parameters, reducing the memory cost of adapting a large language model. During inference, generated moves are normalized and matched against the legal move list before computing top-k predictions. The goal of fine-tuning is not to imitate engine-optimal play, but to learn patterns of likely human decisions conditioned on chess context and player strength. This human-centered formulation is motivated by prior work showing that modeling human chess behavior differs from optimizing purely for engine strength (Tang et al., 2024; Rokach and Shapira, 2026).

Each chess position is converted into a structured prompt containing White Elo, Black Elo, a textual board description, and the legal move list. Chess states and moves are represented in standard plain-text notation based on PGN-style formatting (Feng et al., 2023). By incorporating legal moves directly into the prompt, the model is guided toward ranking plausible candidate actions instead of freely generating unconstrained chess text.

At inference time, the model produces a list of candidate moves using beam search. These predictions are normalized into a consistent label space before being passed to the caching module. For every predicted move, the system computes an engine response in advance and stores it for possible reuse. In this design, the language model is responsible for forecasting likely human behavior, while the chess engine remains responsible for producing strong responses once a likely branch has been identified. This separation allows the system to exploit human predictability without giving up the tactical quality of engine-based play (The Stockfish Developers, 2024; Tang et al., 2024).

4 Experiments

We evaluate whether our Qwen3.5-9B-based predictor can better anticipate human chess moves than engine-based baselines and whether these predictions improve predictive caching. The goal is not to identify the objectively strongest chess move, but to predict the move a human opponent is likely to play. Therefore, we evaluate both move-prediction performance and the resulting reduction in user-perceived latency.

4.1 Dataset and Preprocessing

We use the Lichess Rated Standard Chess Games Dataset, which contains rated human chess games with move sequences, player Elo ratings, game results, time controls, and opening metadata. Each game is represented in Portable Game Notation (PGN). To convert game-level data into supervised prediction examples, we replay each game move by move using a chess library. For each selected position, we extract the current board state, the legal move list, and the opponent's rating information. The prediction label is the next move actually played by the opponent.

We include Elo information because players at different skill levels may prefer different moves in the same position. For example, lower-rated players may prefer forcing captures, checks, or immediate threats, while higher-rated players may choose more positional or opening-theory-based continuations. Including rating information allows the predictor to model likely human behavior rather than only the objective strength of a move.

After filtering, we use 200,000 position-level examples. We split the dataset into training, vali-

dation, and test sets using an 80/10/10 ratio. The split is performed at the game level so that all positions from the same game appear in only one split. This prevents positions from the same game from appearing across both training and testing. The validation set is used for model selection and hyperparameter tuning, while the test set is reserved for final evaluation.

Data Filtering and Stratification: To focus on high-quality prediction examples, we apply the following filtering criteria:

- **Game Resolution and Length Threshold:** We retain only games that end in checkmate and are longer than 35 full moves. This ensures that each retained game contains enough positions for evaluation and allows us to construct position-level prompts from the selected Black-move positions.
- **Full-move Index Selection:** We focus on Black-move predictions from full-move indices 11 through 33. This range avoids the highly memorized opening phase and the simplified endgame, allowing the evaluation to focus on middlegame positions where human move choices are more diverse.
- **Elo Stratification:** To study how player strength affects move predictability, we divide the test examples into three rating buckets based on the opponent's Lichess Elo rating: Easy, Medium, and Hard, corresponding to Elo ranges 0–1199, 1200–1599, and 1600+, respectively. The model is trained on the full training set across all rating levels, while the Elo buckets are used for segmented evaluation and analysis.

4.2 Evaluation Metrics

We evaluate the system using three quantitative metrics.

Prediction Latency. Prediction latency measures the average time required to generate the predicted move set. For the Qwen3.5-9B-based predictor, this corresponds to model inference time; for the Stockfish baselines, it corresponds to engine search time. This metric captures the computational cost of prediction before cache construction.

Cache Hit Rate (CHR). Cache Hit Rate measures how often the true human move appears in the top-k predicted move set. Let $m_t^{(i)}$ denote the true move played in position i, and let $M_k^{(i)}$ denote the set of top-k predicted moves for that position. We define Cache Hit Rate at k as:

$$\text{CHR@}k = \frac{1}{N}\sum_{i=1}^{N} \mathbb{I}[m_t^{(i)} \in M_k^{(i)}],$$

where N is the number of evaluated positions and $\mathbb{I}$ is an indicator function that equals 1 when the true move is included in the predicted set and 0 otherwise.

Average User-Perceived Latency. Average user-perceived latency estimates the delay experienced by the user after the opponent makes a move. Let p be the measured cache hit rate, T_{cache} be the time required to retrieve a precomputed response from the cache, and T_{engine} be the time required to compute a response on demand using the chess engine. The expected user-perceived latency is:

$$\mathbb{E}[L_{\text{perceived}}] = p \times T_{\text{cache}} + (1-p) \times T_{\text{engine}}.$$

This metric measures user-facing latency after prediction and cache construction have completed. It assumes that prediction and response precomputation have already completed before the user needs the system response. Therefore, user-perceived latency only includes cache retrieval time on a cache hit or on-demand engine computation time on a cache miss, while prediction latency is reported separately as a background cost.

4.3 Baselines

We compare our system against one no-cache system baseline and two Stockfish-based prediction baselines.

No-Cache Baseline. The no-cache baseline represents the standard setting where no prediction is performed. The system waits until the opponent's actual move is observed, then calls the chess engine to compute a response on demand. This baseline provides the reference latency for ordinary on-demand engine computation.

Stockfish Top-1 Baseline. The Stockfish top-1 baseline assumes that the opponent will play the strongest engine-recommended move. For each position, Stockfish analyzes the board and returns its highest-ranked move. We then compare this move to the actual human move.

Stockfish Top-k Baseline. The Stockfish top-k baseline extends the previous baseline by using the top-k engine-recommended moves as the predicted move set. We evaluate Stockfish at depth 5 and depth 10. Depth 5 provides a faster but shallower search, while depth 10 provides stronger analysis at a higher computational cost. We obtain the top-k Stockfish moves using Stockfish's MultiPV setting.

4.4 Experimental Design

We structure our evaluation around four experiments.

Experiment 1: LLM vs. Engine Human Modeling by Elo. We train a single predictor on the full training set and then evaluate its CHR across Easy, Medium, and Hard player groups. We compare the predictor against Stockfish depth-5 and depth-10 baselines within each rating bucket. This experiment tests whether a language-model-based predictor captures human move preferences better than an engine that ranks moves by objective strength, and whether prediction difficulty changes with player rating.

Experiment 2: Prediction Latency. We measure the average time required for each method to generate a ranked predicted move list. This includes the inference time of the Qwen3.5-9B-based predictor and the search time required by the Stockfish depth-5 and depth-10 baselines. Because each method produces a ranked candidate list in a single call, prediction latency is reported once per method rather than separately for each value of k. This experiment evaluates the computational cost of producing predictions before cache construction.

Experiment 3: User-Perceived Latency with Predictive Caching. We estimate the reduction in user-facing latency achieved by predictive caching across prediction set sizes $k \in \{1, 3, 5\}$. For each method, including the Qwen3.5-9B-based predictor and the Stockfish baselines, the predicted top-k moves are used as cache candidates. If the true opponent move appears in the predicted set, the system retrieves a precomputed response from the cache; otherwise, it falls back to on-demand engine computation. In this experiment, T_{engine} is measured as the time required to compute an on-demand Stockfish response after the actual opponent move is observed. We set the cache lookup time as $T_{\text{cache}} = 0.05\,\text{ms}$ and combine this value with each method's CHR and the measured T_{engine} to estimate user-perceived latency. This experiment studies how prediction accuracy translates into practical latency reduction for the user.

Experiment 4: Cache Hit Rate with Full-move Index Analysis. To assess model performance across evolving game stages, we track CHR continuously from full-move indices 11 through 33. This experiment identifies how the relative performance of the language-model-based predictor and engine baselines changes across the game, and where each method tends to fail.

5 Results and Discussion

Our experiments evaluate whether a fine-tuned LLM can predict human chess moves more accurately than Stockfish baselines, and whether these predictions can support predictive caching.

5.1 Experimental Results

Experiment 1: LLM vs. Engine Human Modeling by Elo. Figure 1 illustrates the Cache Hit Rate of the Qwen3.5-9B model compared to Stockfish baselines at search depths of 5 and 10. The performance is evaluated across three player skill brackets: Easy, Medium, and Hard, alongside an Overall average. The results are further broken down by prediction set sizes of top-1, top-3, and top-5. Across all skill brackets and the Overall average, the Qwen3.5-9B model achieves a higher cache hit rate than both Stockfish baselines for top-1 and top-3 predictions. For top-5 predictions, the margin between Qwen3.5-9B and the Stockfish baselines is smaller. Qwen3.5-9B maintains the highest cache hit rate for top-5 predictions in the Easy and Medium categories, as well as in the Overall average. In the Hard category, Stockfish at depth 10 achieves a slightly higher cache hit rate than Qwen3.5-9B for top-5 predictions.

Experiment 2: Prediction Latency. Figure 2 displays the average prediction latency, measured in milliseconds (ms), for the Qwen3.5-9B model alongside two Stockfish baselines evaluated at depth 5 and depth 10. Among the three evaluated configurations, the Qwen3.5-9B model shows the highest average prediction latency, positioned above the 250 ms mark at approximately 264 ms. Stockfish at depth 10 displays the second-highest average latency, registering just below the 200 ms mark at approximately 196 ms. Stockfish at depth 5 exhibits the lowest average prediction latency of

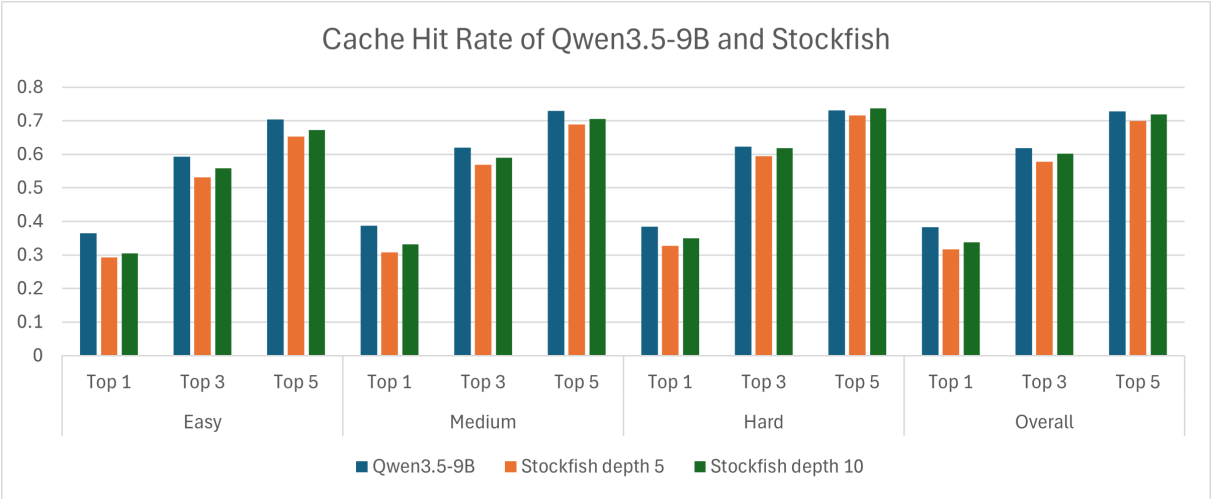

Figure 1: Cache Hit Rate of Qwen3.5-9B and Stockfish baselines by Elo bucket

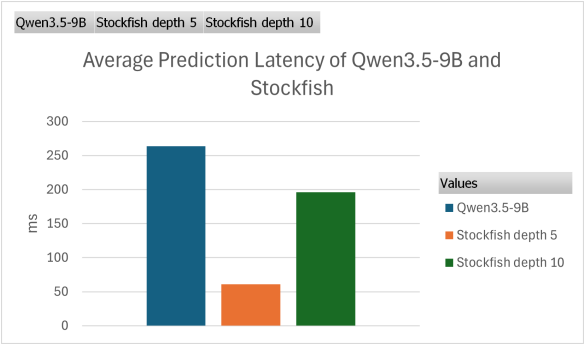

Figure 2: Average prediction latency of Qwen3.5-9B and Stockfish baselines

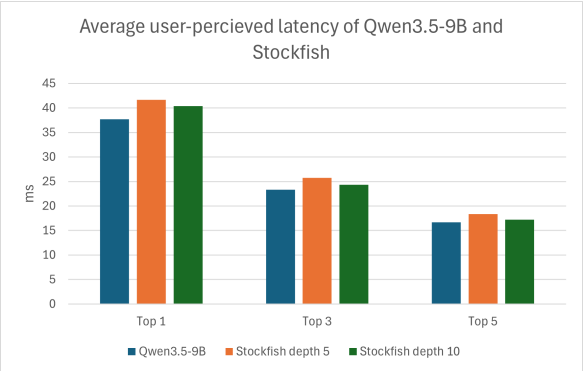

Figure 3: Average user-perceived latency of Qwen3.5-9B and Stockfish

the group, measuring just above the 50 ms mark at approximately 61 ms.

Experiment 3: User-Perceived Latency with Predictive Caching. The no-cache baseline corresponds to computing the Stockfish response only after the actual opponent move is observed, so its latency is equivalent to T_{engine} and serves as the reference point for measuring the benefit of cache hits. Figure 3 displays the average user-perceived latency, measured in milliseconds (ms), for the Qwen3.5-9B model compared to Stockfish baselines at depth 5 and depth 10. Across all three prediction categories, the Qwen3.5-9B model consistently exhibits the lowest average user-perceived latency. For top-1 predictions, Qwen3.5-9B records a latency below 40 ms, while Stockfish at depth 5 shows the highest latency in this category at over 40 ms. As the prediction set size expands to top-3 and top-5, the user-perceived latency decreases across all models. By the top-5 category, the average latencies for all three configurations drop below 20 ms, with Qwen3.5-9B remaining the lowest.

Experiment 4: Cache Hit Rate with Full-move Index Analysis. Figure 4 displays the Cache Hit Rate of the Qwen3.5-9B model alongside Stockfish evaluated at depth 5 and depth 10, plotted across a sequence of full-move indices from 11 to 33. Within the top-1 prediction group, Qwen3.5-9B begins with the highest cache hit rate at full-move index 11 but exhibits a downward trend as the full-move index increases. The Stockfish baselines in the top-1 group remain relatively stable, leading to a convergence of all three models by full-move index 33. For top-3 and top-5 predictions, Qwen3.5-9B similarly starts with the highest cache hit rates at earlier full-move indices but steadily declines as the game progresses. In these larger prediction sets, the Stockfish baselines maintain more constant hit rates, with Stockfish depth 10 eventually surpassing Qwen3.5-9B in the later stages of the evaluated full-move indices.

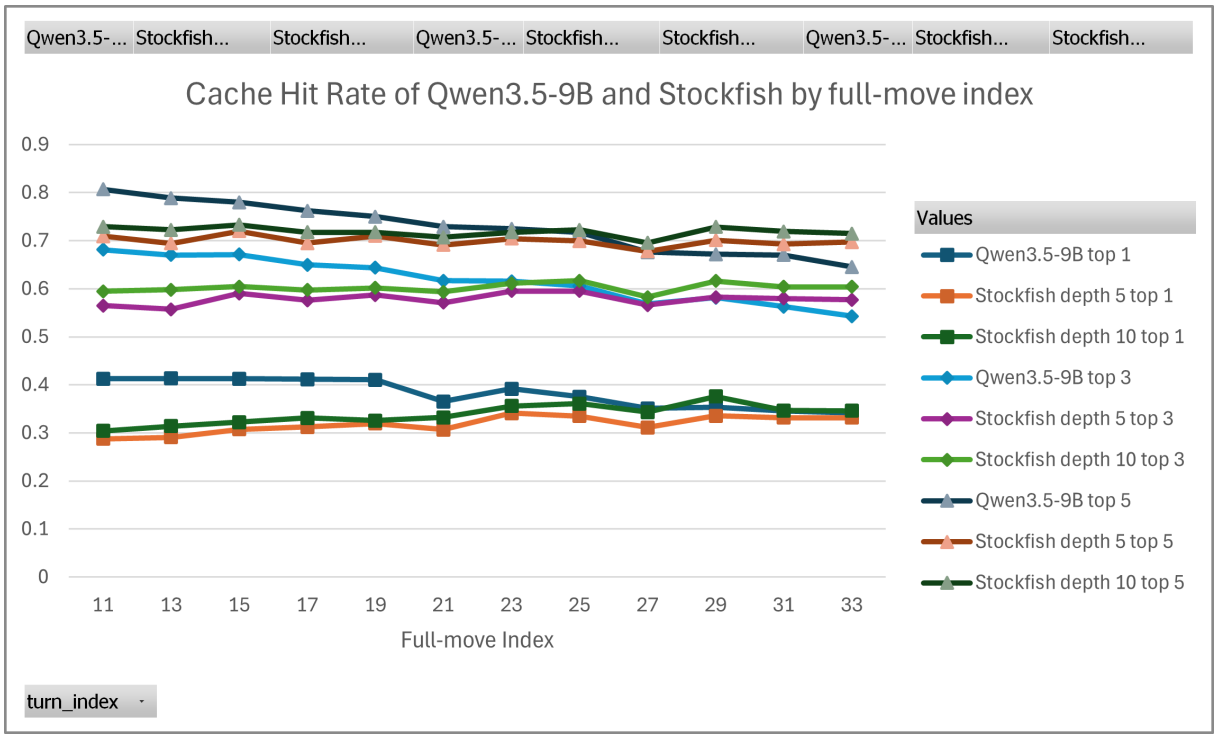

Figure 4: Cache Hit Rate of Qwen3.5-9B and Stockfish baselines by full-move index

5.2 Discussion

The experimental results demonstrate a fundamental distinction between computing the objectively best chess move and predicting human behavior. The consistent superiority of the Qwen3.5-9B model in top-1 and top-3 cache hit rates across varying player skill levels indicates that the fine-tuned language model successfully captures human playing styles, common heuristics, and even predictable inaccuracies. By prioritizing moves that human players are actually likely to make, rather than strictly optimal lines, the LLM creates a much more effective predictive cache.

A critical insight from this study is the successful decoupling of prediction latency from user-perceived latency. The latency results reveal a significant computational tradeoff: Qwen3.5-9B introduces a prediction-time bottleneck compared with the Stockfish baselines, taking roughly four times longer to generate predictions than Stockfish at depth 5. However, because the LLM is highly accurate at predicting the opponent's actual move, the system frequently bypasses this bottleneck via cache hits. The resulting drop in user-perceived latency—falling below 20 ms for top-5 predictions—suggests that high-latency but accurate predictive models can still be useful when their predictions are computed in the background before the user needs a response.

Despite these advantages, the full-move-index analysis highlights a distinct failure mode for the language model as the game progresses. While the LLM dominates in the early to mid-game stages, its accuracy steadily declines, leading to a convergence where Stockfish eventually surpasses it in top-3 and top-5 predictions. This degradation likely occurs because early-game positions rely heavily on recognizable structural patterns and standard developmental ideas, which language models excel at memorizing and reproducing. Conversely, late-game positions become highly open, tactical, and calculative. In these scenarios, human players—especially in the Hard Elo bracket, where Stockfish depth 10 performed best for top-5 predictions—often find forced tactical lines that align closely with traditional engine search trees.

These findings suggest that a pure LLM approach is suboptimal for a complete game lifecycle due to both its late-game accuracy drop and its higher computational cost relative to Stockfish. A highly effective future architecture would be a stage-aware hybrid ranker. Such a system could leverage the LLM's strong human-modeling capabilities during the early and middle stages of the game, and dynamically route prediction tasks to an engine like Stockfish in later-game or highly tacti-

cal positions where deep search is strictly superior. Because prediction and response precomputation complete in less than one second in our experiments, the cache can usually be prepared before the next system response is needed. This supports the separation between prediction latency as a background cost and user-perceived latency as the delay experienced after the actual move is observed.

6 Conclusion

In this project, we studied whether human next-move prediction can support predictive caching in chess systems. We framed the task as top-k human move forecasting conditioned on board state, legal moves, and player metadata, and implemented a legality-aware QLoRA fine-tuning pipeline using Qwen3.5-9B. Our results show that language-model-based prediction can better capture human move choices in some settings, especially for top-1 prediction and earlier game stages, while Stockfish remains competitive or stronger in later positions. We also found that cache hit rates were relatively consistent across Elo buckets, suggesting that the approach generalizes reasonably well across different player skill levels.

Beyond move prediction, our results show that predictive caching can improve system responsiveness by precomputing responses for likely human moves. However, the language model introduces a prediction-time bottleneck relative to the Stockfish baselines, even though this cost can be treated as background computation when precomputation finishes before the next response is needed. This suggests that future work should explore faster inference methods and stage-aware hybrid systems that use language-model ranking in earlier positions and engine-based ranking in later or more tactical positions. Overall, this project shows that human move prediction is useful not only for modeling player behavior, but also for improving responsiveness in proactive intelligent systems.

7 Acknowledgment

We acknowledge that artificial intelligence (AI) was used as a tool to assist in the research, drafting, and editing of this work.

References

Xidong Feng, Yicheng Luo, Ziyan Wang, Hongrui Tang, Mengyue Yang, Kun Shao, David Mguni, Yali Du, and Jun Wang. 2023. Chessgpt: Bridging policy learning and language modeling. *Preprint*, arXiv:2306.09200.

Gregory Gutmann and Akihiko Konagaya. 2019. Predictive simulation: Using regression and artificial neural networks to negate latency in networked interactive virtual reality. *Preprint*, arXiv:1910.04703.

David Halbhuber, Maximilian Seewald, Fabian Schiller, Mathias Götz, Jakob Fehle, and Niels Henze. 2022. Using artificial neural networks to compensate negative effects of latency in commercial real-time strategy games. In *Proceedings of Mensch Und Computer 2022*, MuC '22, page 182–191, New York, NY, USA. Association for Computing Machinery.

Sai Kolasani, Maxim Saplin, Nicholas Crispino, Kyle Montgomery, Jared Quincy Davis, Matei Zaharia, Chi Wang, and Chenguang Wang. 2025. Llm chess: Benchmarking reasoning and instruction-following in llms through chess. *Preprint*, arXiv:2512.01992.

Benjamin Kreiger. 2024. Predicting human chess moves with large language models. Master's thesis, University of South Florida.

Alessandro Libralesso. 2025. Shashguru: Bridging chess engines and large language models for human-interpretable analysis. Master's thesis, Università di Bologna.

Takato Motoo, Jiei Kawasaki, Takuya Fujihashi, Shunsuke Saruwatari, and Takashi Watanabe. 2021. Client-side network delay compensation for online shooting games. *IEEE Access*, 9:125678–125690.

Yarden Rokach and Bracha Shapira. 2026. Blunder prediction in chess. *Applied Intelligence*, 56(4).

David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. 2017. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *Preprint*, arXiv:1712.01815.

Zhenwei Tang, Difan Jiao, Reid McIlroy-Young, Jon Kleinberg, Siddhartha Sen, and Ashton Anderson. 2024. Maia-2: A unified model for human-ai alignment in chess. *Preprint*, arXiv:2409.20553.

The Stockfish Developers. 2024. Stockfish: A free and strong open-source UCI chess engine. `https://stockfishchess.org/`. Version 16.1.